

Total Marks: 2.5

Obtained Marks: _____

DSA Lab

Task # 01

Last Date of Submission: 6th October 2026

Submitted To: Mr. Muhammad Waqas

Student Name: Anique Ali

Registration Number: 25108113

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

***Instructions:** Copied or shown assignments will be marked zero. Late submissions are not entertained in any case.*

CLO 3 – C2 – PLO C, E

Q1: Hospital Emergency Patient Queue

A hospital emergency department needs a simple system to manage patients waiting for treatment. Each patient has a **patient ID** and **name**.

The hospital wants to use a **singly linked list** to store patients in the order they arrive.

The system should provide the following operations:

1. **Add a patient** at the end of the list.
2. **Display all patients** currently waiting.
3. **Search for a patient** using their ID.
4. **Remove a patient** when they have been treated.
5. **Count the total number of patients** waiting.

Source Code:

```
#include <iostream>

#include <string>

using namespace std;

struct Node {

    int id;

    string name;

    Node* next;

};

Node* head = NULL;

void addPatient() {

    Node* newNode = new Node;
```

```
cout << "Enter Patient ID: ";

cin >> newNode->id;


cout << "Enter Patient Name: ";

cin >> ws;

getline(cin, newNode->name);


newNode->next = NULL;


if (head == NULL) {

    head = newNode;

}

else {

    Node* temp = head;

    while (temp->next != NULL) {

        temp = temp->next;

    }

    temp->next = newNode;

}

cout << "Patient added successfully.\n";

}

void displayPatients() {

    if (head == NULL) {

        cout << "No patients waiting.\n";
```

```
        return;

    }

    Node* temp = head;

    while (temp != NULL) {

        cout << "ID: " << temp->id;

        cout << " Name: " << temp->name << endl;

        temp = temp->next;

    }

}

void searchPatient() {

    int id;

    cout << "Enter Patient ID: ";

    cin >> id;

    Node* temp = head;

    while (temp != NULL) {

        if (temp->id == id) {

            cout << "Patient Found: "

                << temp->name << endl;

            return;

        }

        temp = temp->next;

    }

    cout << "Patient not found.\n";

}

void removePatient() {
```

```
int id;

cout << "Enter Patient ID to remove: ";

cin >> id;

if (head == NULL) {

    cout << "List is empty.\n";

    return;

}

if (head->id == id) {

    Node* temp = head;

    head = head->next;

    delete temp;

    cout << "Patient removed.\n";

    return;

}

Node* temp = head;

while (temp->next != NULL &&

    temp->next->id != id) {

    temp = temp->next;

}

if (temp->next == NULL) {

    cout << "Patient not found.\n";

}

else {

    Node* del = temp->next;
```

```
temp->next = del->next;

delete del;

cout << "Patient removed.\n";

}

}

void countPatients() {

    int count = 0;

    Node* temp = head;

    while (temp != NULL) {

        count++;

        temp = temp->next;

    }

    cout << "Total waiting patients: "

        << count << endl;

}

int main() {

    int choice;

    do {

        cout << "\nHospital Patient Management\n";

        cout << "1. Add Patient\n";

        cout << "2. Display Patients\n";

        cout << "3. Search Patient\n";

        cout << "4. Remove Patient\n";

        cout << "5. Count Patients\n";
```

```
cout << "6. Exit\n";

cout << "Enter choice: ";

cin >> choice;

switch (choice) {

    case 1:

        addPatient();

        break;

    case 2:

        displayPatients();

        break;

    case 3:

        searchPatient();

        break;

    case 4:

        removePatient();

        break;

    case 5:

        countPatients();

        break;

    case 6:

        cout << "Exiting program.\n";

        break;

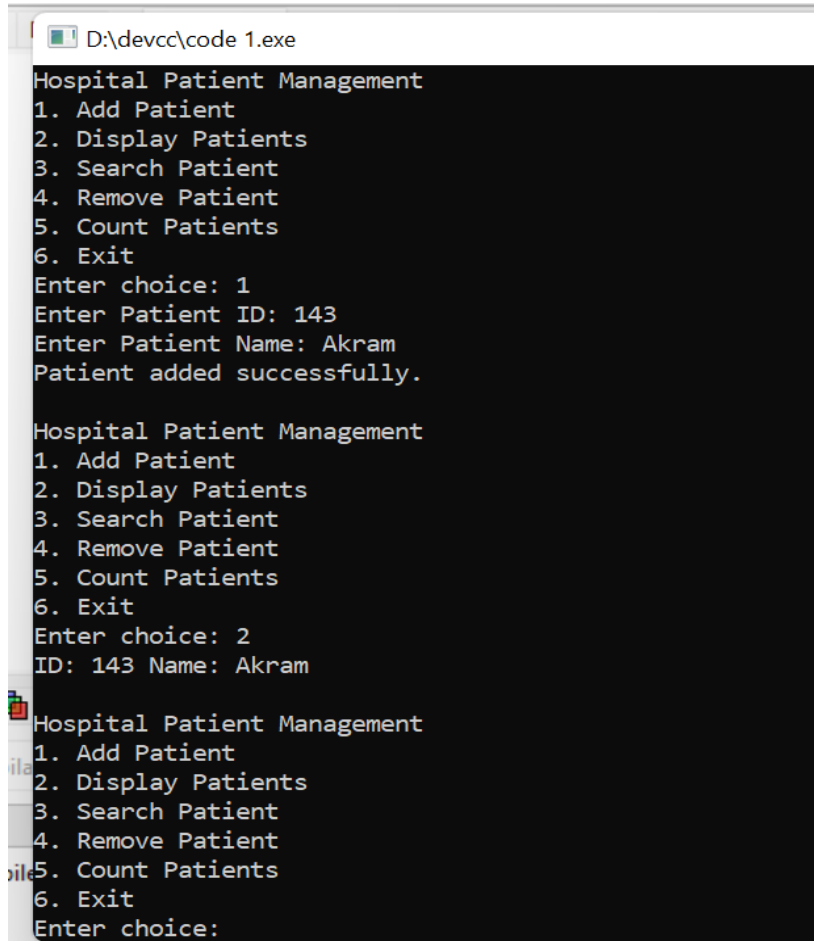
    default:

        cout << "Invalid choice.\n";

}
```

```
} while (choice != 6);  
  
return 0;  
  
}
```

Output:



```
D:\devcc\code 1.exe  
Hospital Patient Management  
1. Add Patient  
2. Display Patients  
3. Search Patient  
4. Remove Patient  
5. Count Patients  
6. Exit  
Enter choice: 1  
Enter Patient ID: 143  
Enter Patient Name: Akram  
Patient added successfully.  
  
Hospital Patient Management  
1. Add Patient  
2. Display Patients  
3. Search Patient  
4. Remove Patient  
5. Count Patients  
6. Exit  
Enter choice: 2  
ID: 143 Name: Akram  
  
Hospital Patient Management  
1. Add Patient  
2. Display Patients  
3. Search Patient  
4. Remove Patient  
5. Count Patients  
6. Exit  
Enter choice:
```

Q2: Airport Flight Management System

An **airport flight management system** stores flight records using a **doubly linked list**. Each flight record contains a **Flight ID, Destination, and Departure Time**. Each node has two pointers:

- `next` → points to the next flight
- `prev` → points to the previous flight

The system should provide the following operations:

1. Add a new flight at the end of the list.
2. Display all flights from **first to last**.
3. Display all flights from **last to first**.
4. Search for a flight using a **Flight ID entered by the user**.
5. Delete a flight using a **Flight ID entered by the user**.
6. Count the total number of flights.

Write a C++ program using a **doubly linked list** to implement this airport flight management system.

Source Code:

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    int flightID;
    string destination;
    string departureTime;
    Node* next;
    Node* prev;
};

Node* head = NULL;
Node* tail = NULL;

void addFlight() {
    Node* newNode = new Node;

    cout << "Enter Flight ID: ";
    cin >> newNode->flightID;

    cout << "Enter Destination: ";
    cin >> ws;
    getline(cin, newNode->destination);

    cout << "Enter Departure Time: ";
    getline(cin, newNode->departureTime);
```

```
newNode->next = NULL;
newNode->prev = NULL;

if (head == NULL) {
    head = newNode;
    tail = newNode;
}
else {
    tail->next = newNode;
    newNode->prev = tail;
    tail = newNode;
}

cout << "Flight added successfully.\n";
}

void displayForward() {
    if (head == NULL) {
        cout << "No flights available.\n";
        return;
    }

    Node* temp = head;

    while (temp != NULL) {
        cout << "Flight ID: " << temp->flightID << endl;
        cout << "Destination: " << temp->destination << endl;
        cout << "Departure Time: "
            << temp->departureTime << endl;

        temp = temp->next;
    }
}

void displayBackward() {
    if (tail == NULL) {
        cout << "No flights available.\n";
        return;
    }

    Node* temp = tail;

    while (temp != NULL) {
        cout << "Flight ID: " << temp->flightID << endl;
        cout << "Destination: " << temp->destination << endl;
        cout << "Departure Time: "
            << temp->departureTime << endl;

        temp = temp->prev;
    }
}

void searchFlight() {
    int id;
```

```
cout << "Enter Flight ID to search: ";
cin >> id;

Node* temp = head;

while (temp != NULL) {
    if (temp->flightID == id) {
        cout << "Flight Found!\n";
        cout << "Destination: "
            << temp->destination << endl;
        cout << "Departure Time: "
            << temp->departureTime << endl;
        return;
    }

    temp = temp->next;
}

cout << "Flight not found.\n";
}

void deleteFlight() {
    int id;

    cout << "Enter Flight ID to delete: ";
    cin >> id;

    Node* temp = head;

    while (temp != NULL && temp->flightID != id) {
        temp = temp->next;
    }

    if (temp == NULL) {
        cout << "Flight not found.\n";
        return;
    }

    if (temp == head) {
        head = temp->next;
    }

    if (temp == tail) {
        tail = temp->prev;
    }

    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }

    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
}
```

```
delete temp;

cout << "Flight deleted successfully.\n";
}

void countFlights() {
    int count = 0;
    Node* temp = head;

    while (temp != NULL) {
        count++;
        temp = temp->next;
    }

    cout << "Total Flights: " << count << endl;
}

int main() {
    int choice;

    do {
        cout << "\nAirport Flight Management System\n";
        cout << "1. Add Flight\n";
        cout << "2. Display Forward\n";
        cout << "3. Display Backward\n";
        cout << "4. Search Flight\n";
        cout << "5. Delete Flight\n";
        cout << "6. Count Flights\n";
        cout << "7. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                addFlight();
                break;

            case 2:
                displayForward();
                break;

            case 3:
                displayBackward();
                break;

            case 4:
                searchFlight();
                break;

            case 5:
                deleteFlight();
                break;

            case 6:
```

```
        countFlights();
        break;

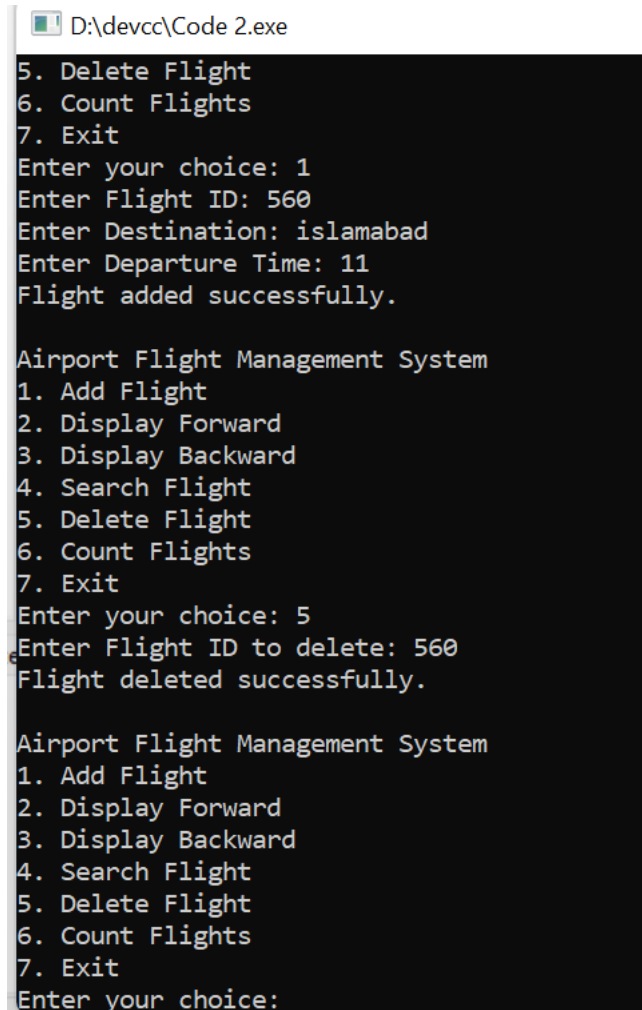
    case 7:
        cout << "Exiting program.\n";
        break;

    default:
        cout << "Invalid choice.\n";
    }

} while (choice != 7);

return 0;
}
```

Output:



```
D:\devcc\Code 2.exe
5. Delete Flight
6. Count Flights
7. Exit
Enter your choice: 1
Enter Flight ID: 560
Enter Destination: islamabad
Enter Departure Time: 11
Flight added successfully.

Airport Flight Management System
1. Add Flight
2. Display Forward
3. Display Backward
4. Search Flight
5. Delete Flight
6. Count Flights
7. Exit
Enter your choice: 5
Enter Flight ID to delete: 560
Flight deleted successfully.

Airport Flight Management System
1. Add Flight
2. Display Forward
3. Display Backward
4. Search Flight
5. Delete Flight
6. Count Flights
7. Exit
Enter your choice:
```

Q3: Parcel Delivery Management System

A courier company maintains its pending parcels in a **Singly Linked List**. Each parcel contains:

- Parcel ID
- Customer Name
- Destination
- Next parcel address

The parcels are stored in the order in which they are received. The company wants a program that allows the staff to manage the parcel list.

Requirements

The program should provide a menu with the following options:

1. **Add Parcel**
 - Take parcel information from the user.
 - Add the new parcel at the **end** of the linked list.
2. **Display All Parcels**
 - Display all parcels from **HEAD** to **NULL**.
3. **Search Parcel**
 - Ask the user to enter a Parcel ID.
 - Search the linked list and display the parcel details if found.
4. **Delete Parcel**
 - Ask the user for a Parcel ID.
 - Remove that parcel from the linked list.
 - Correctly handle deletion of:
 - First node
 - Middle node
 - Last node
5. **Count Parcels**
 - Display the total number of parcels currently in the list.
6. **Add Urgent Parcel**
 - Add an urgent parcel at the **beginning** of the list.
 - This demonstrates insertion at **HEAD**.
7. **Exit**

The program should use **dynamic memory allocation** and pointers.

Source Code:

```
#include <iostream>
#include <string>
using namespace std;
```

```
struct Node {
    int parcelID;
    string customerName;
    string destination;
    Node* next;
```

};

Node* head = NULL;

void addParcel() {

Node* newNode = new Node;

cout << "Enter Parcel ID: ";

cin >> newNode->parcelID;

cout << "Enter Customer Name: ";

cin >> ws;

getline(cin, newNode->customerName);

cout << "Enter Destination: ";

getline(cin, newNode->destination);

newNode->next = NULL;

if (head == NULL) {

head = newNode;

}

else {

Node* temp = head;

while (temp->next != NULL) {

temp = temp->next;

}

temp->next = newNode;

}

cout << "Parcel added successfully.\n";

}

void displayParcels() {

if (head == NULL) {

cout << "No parcels available.\n";

return;

}

Node* temp = head;

while (temp != NULL) {

cout << "\nParcel ID: " << temp->parcelID;

cout << "\nCustomer Name: " << temp->customerName;

cout << "\nDestination: " << temp->destination;

cout << endl;

temp = temp->next;

}

}

```
void searchParcel() {
    int id;

    cout << "Enter Parcel ID to search: ";
    cin >> id;

    Node* temp = head;

    while (temp != NULL) {
        if (temp->parcelID == id) {
            cout << "Parcel Found!\n";
            cout << "Parcel ID: " << temp->parcelID << endl;
            cout << "Customer Name: " << temp->customerName << endl;
            cout << "Destination: " << temp->destination << endl;
            return;
        }

        temp = temp->next;
    }

    cout << "Parcel not found.\n";
}
```

```
void deleteParcel() {
    int id;

    cout << "Enter Parcel ID to delete: ";
    cin >> id;

    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }

    if (head->parcelID == id) {
        Node* temp = head;
        head = head->next;
        delete temp;

        cout << "Parcel deleted successfully.\n";
        return;
    }

    Node* temp = head;

    while (temp->next != NULL &&
        temp->next->parcelID != id) {
        temp = temp->next;
    }
}
```

```
    if (temp->next == NULL) {
        cout << "Parcel not found.\n";
    }
    else {
        Node* del = temp->next;
        temp->next = del->next;
        delete del;

        cout << "Parcel deleted successfully.\n";
    }
}

void countParcels() {
    int count = 0;
    Node* temp = head;

    while (temp != NULL) {
        count++;
        temp = temp->next;
    }

    cout << "Total Parcels: " << count << endl;
}

void addUrgentParcel() {
    Node* newNode = new Node;

    cout << "Enter Urgent Parcel ID: ";
    cin >> newNode->parcelID;

    cout << "Enter Customer Name: ";
    cin >> ws;
    getline(cin, newNode->customerName);

    cout << "Enter Destination: ";
    getline(cin, newNode->destination);

    newNode->next = head;
    head = newNode;

    cout << "Urgent parcel added successfully.\n";
}

int main() {
    int choice;

    do {
        cout << "\nParcel Delivery Management System\n";
        cout << "1. Add Parcel\n";
        cout << "2. Display All Parcels\n";
```

```
cout << "3. Search Parcel\n";
cout << "4. Delete Parcel\n";
cout << "5. Count Parcels\n";
cout << "6. Add Urgent Parcel\n";
cout << "7. Exit\n";

cout << "Enter your choice: ";
cin >> choice;

switch (choice) {
    case 1:
        addParcel();
        break;

    case 2:
        displayParcels();
        break;

    case 3:
        searchParcel();
        break;

    case 4:
        deleteParcel();
        break;

    case 5:
        countParcels();
        break;

    case 6:
        addUrgentParcel();
        break;

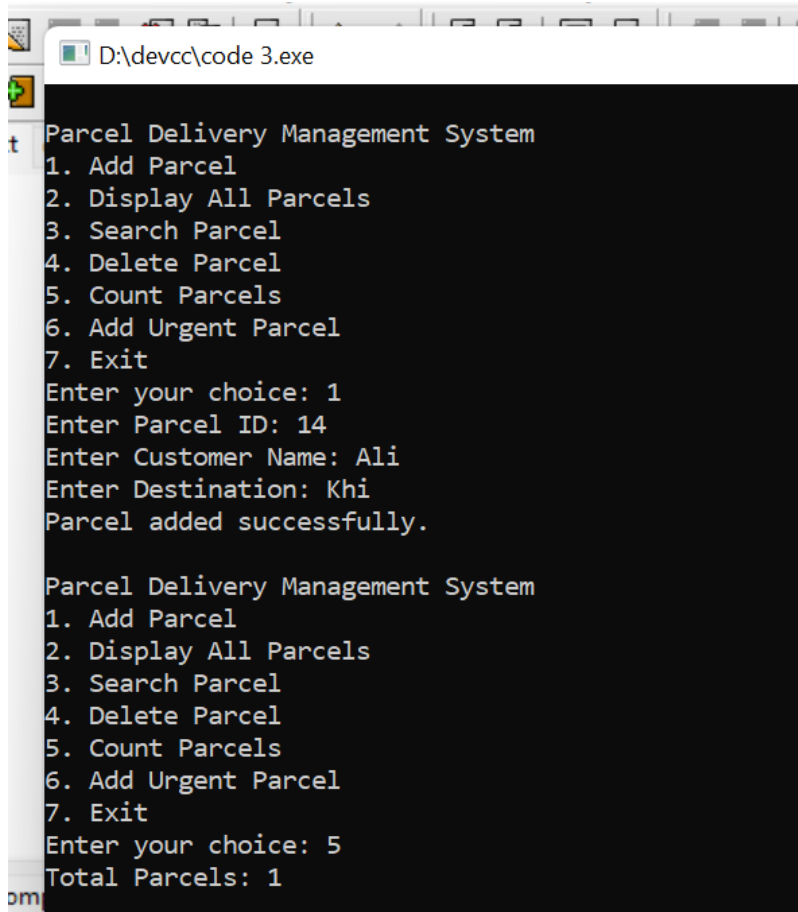
    case 7:
        cout << "Exiting program.\n";
        break;

    default:
        cout << "Invalid choice.\n";
}

} while (choice != 7);

return 0;
}
```

Output:



```
D:\devcc\code 3.exe

Parcel Delivery Management System
1. Add Parcel
2. Display All Parcels
3. Search Parcel
4. Delete Parcel
5. Count Parcels
6. Add Urgent Parcel
7. Exit
Enter your choice: 1
Enter Parcel ID: 14
Enter Customer Name: Ali
Enter Destination: Khi
Parcel added successfully.

Parcel Delivery Management System
1. Add Parcel
2. Display All Parcels
3. Search Parcel
4. Delete Parcel
5. Count Parcels
6. Add Urgent Parcel
7. Exit
Enter your choice: 5
Total Parcels: 1
```